Messaging

Messaggistica (messaging)

I componenti distribuiti o le varie applicazioni devono comunicare attraverso la rete, la quale è inaffidabile (poiché può causare ritardi o interruzioni) e lenta rispetto a una comunicazione locale. Poiché le applicazioni usano linguaggi di programmazione e formati di dati diversi, è necessaria una soluzione di integrazione per poter interfacciare queste tecnologie. La messaggistica è una tecnologia che abilita comunicazioni asincrone fra programmi con consegne affidabili. I programmi comunicano inviando pacchetti di dati chiamati messaggi, che viaggiano attraverso percorsi logici definiti canali (o code).

Obiettivi

Un messaggio contiene un certo tipo di dato (stringa, byte, record, oggetto) ed è composto da due parti:

1. un **header** (che contiene informazioni sul messaggio, come mittente e destinatario).
2. un **body** (che contiene i dati trasmessi).

Le architetture di messaggistica asincrona sono molto potenti grazie all'uso di un middleware di messaggistica che coordina e gestisce l'invio e la ricezione in modo affidabile. Poiché la rete può fallire nel mandare i dati, il sistema di messaggistica supera questa inaffidabilità ritrasmettendo il messaggio finché non ci riesce.

Vantaggi dei messaggi

L'adozione della messaggistica offre diversi benefici strutturali:

- **Abilita la comunicazione e il trasferimento dati**: permette di copiare, serializzare, e inviare oggetti attraverso la rete ad altri host, a differenza di quanto avviene nello stesso processo dove i dati si condividono in memoria.
- **Integrazione di piattaforme e linguaggi**: il sistema di messaggistica funge da traduttore universale fra applicazioni che si basano su tecnologie diverse.
- **Comunicazione asincrona**: permette un approccio *send and forget* in cui chi invia non deve aspettare che il destinatario riceva o elabori il messaggio, né deve aspettare il sistema di invio. Il mittente può svolgere altre operazioni in background e gestire l'eventuale risposta tramite un meccanismo di callback.
- **Temporizzazione variabile**: a differenza delle comunicazioni sincrone (il chiamante si deve adattare alla velocità di elaborazione del destinatario), il richiedente può mandare richieste alla propria velocità e il ricevente le consumerà seguendo il proprio ritmo.
- **Gestione di tante chiamate**: la comunicazione asincrona evita il sovraccarico del ricevente, permettendogli di controllare la frequenza di consumo delle richieste in ingresso.
- **Comunicazione affidabile**: utilizza un solido approccio di tipo *store and forward* per la gestione dei messaggi.

RabbitMQ

RabbitMQ è un **broker di messaggi** e un **server di code** che viene usato dalle applicazioni per condividere dati attraverso un protocollo comune o per accodare lavori. Implementa lo standard aperto **AMQP** (*Advanced Message Queuing Protocol*) e agisce da intermediario fra l'applicazione e il server. Il suo funzionamento si basa sui seguenti elementi:

- **Produttori**: creano i messaggi e li pubblicano (inviano) al broker. Ogni messaggio ha un **payload** (cioè i dati da trasferire) e una **label** (che descrive il payload e viene usata da RabbitMQ per determinare a chi mandare il messaggio). La loro comunicazione è *fire-and-forget* e unidirezionale, cioè chi manda non si aspetta una risposta.
- **Consumatori**: si collegano al broker e si sottoscrivono a una coda per attendere i messaggi. Ricevono esclusivamente il payload, senza conoscere la label o l'identità del Produttore.
- **Canali AMQP**: sono connessioni virtuali create dall'applicazione all'interno della connessione TCP. Ogni canale ha un ID univoco e viene usato per pubblicare, sottoscrivere o ricevere messaggi, permettendo di risparmiare le risorse e i tempi necessari per creare molteplici connessione TCP reali.
- **Architettura di invio**: per mandare un messaggio si usano tre componenti:
  - gli **scambi** (*exchange*, dove i produttori pubblicano i messaggi).
  - le **code** (*queue*, dove i messaggi aspettano i consumatori). I consumatori ricevono messaggi da una coda su cui sono iscritti o se richiedono un singolo messaggio da una coda.
  - le **connessioni** (*binding*, che permettono ai messaggi di viaggiare dallo scambio alla coda corretta).

  Se una coda ha uno o più iscritti (consumatori), i messaggi vengono mandati immediatamente, altrimenti restano in attesa.

Code di RabbitMQ

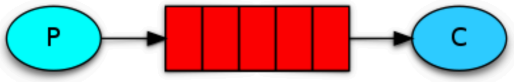
Le code sono elementi fondamentali del sistema a cui sia i consumatori che i produttori possono provvedere alla creazione (tuttavia, un consumatore non può crearne una se è già iscritto a un'altra coda sullo stesso canale). Il nome fornito durante la creazione permette ai consumatori di iscriversi. Le code svolgono tre funzioni principali:

1. Rappresentano il punto di arrivo dei messaggi.
2. Forniscono un posto dove i messaggi aspettano di essere consumati.
3. Forniscono un mezzo per il bilanciamento del carico: se ci sono tanti consumatori iscritti a una singola coda, RabbitMQ distribuisce loro i messaggi in modalità round-robin (ogni messaggio viene inviato a un solo consumatore).

Per gestire la ricezione, il consumatore invia un **ack (ricevuta)** a RabbitMQ, che provvede poi a rimuovere il messaggio dalla coda. Se il consumatore non manda l'ack (perché magari ha rallentato intenzionalmente l'invio per non sovraccaricarsi), RabbitMQ lo considera "non pronto" e sospende l'invio di ulteriori messaggi. Nel caso in cui il consumatore si disconnetta prima di mandare l'ack, o rifiuti esplicitamente il messaggio (e il parametro di *requeue* è true), RabbitMQ provvede a inoltrare il messaggio a un altro consumatore. Mentre, se il parametro requeue è false il messaggio viene scartato.

Esempio 1

Questo è l'esempio base di interazione: vi è un **produttore (P)** che manda un messaggio diretto a una certa coda, e un **consumatore (C)** che riceve il messaggio non appena questo arriva sulla coda.



Produttore

Il produttore ha il compito di generare e inviare il messaggio (ad esempio una stringa). Il suo flusso di esecuzione prevede di:

- **Connettersi a RabbitMQ**: crea una ConnectionFactory, imposta l'host (es. "localhost"), crea la connessione e infine crea un canale.
- **Dichiara la coda**: utilizza il metodo queueDeclare() specificando il nome della coda e altri parametri (se è durable, exclusive o autoDelete):
  - *durable*: sopravvive al riavvio del server.
  - *exclusive*: solo per questa connessione.
  - *autoDelete*: cancellabile dal server se non usata.
- **Pubblicare il messaggio**: utilizza il metodo basicPublish() indicando l'exchange (vuoto nell'esempio), la routing key (che corrisponde al nome della coda), le properties e il body (il contenuto del messaggio convertito in byte).

```
ConnectionFactory factory = new ConnectionFactory();
factory.setHost("localhost"); // nome della macchina del broker
Connection connection = factory.newConnection(); // crea connessione
Channel channel = connection.createChannel(); // crea canale

String QUEUE_NAME = "queue-hello";

channel.queueDeclare(QUEUE_NAME, false, false, false, null); // crea coda

channel.basicPublish("", QUEUE_NAME, null, "hello".getBytes());//invia hel
lo
```

Consumatore

Il consumatore si occupa di mettersi in attesa e leggere i messaggi. Per farlo:

- Crea la connessione, il canale e dichiara la coda **esattamente in modo analogo** a quanto fatto dal produttore.
- **Registra un metodo di callback**: istanzia un oggetto di tipo Consumer (spesso estendendo DefaultConsumer) e ne sovrascrive il metodo handleDelivery(). Questo metodo contiene la logica che verrà eseguita automaticamente quando un messaggio si rende disponibile (ad esempio, estrarre e stampare la stringa dal body).

- **Avvia l'ascolto**: utilizza il metodo basicConsume() sul canale, passandogli il nome della coda, un flag per l'auto-ack e l'oggetto consumatore appena creato.

```
ConnectionFactory factory = new ConnectionFactory();
factory.setHost("localhost"); // nome della macchina del broker
Connection connection = factory.newConnection(); // crea connessione
Channel channel = connection.createChannel(); // crea canale

String QUEUE_NAME = "queue-hello";

Consumer consumer = new DefaultConsumer(channel) {
        @Override // metodo di callback
        public void handleDelivery(String consumerTag, Envelope envelope,
                        AMQP.BasicProperties properties, byte[] body) thro
ws IOException {
                String message = new String(body, "UTF-8");
                System.out.println("Ricevuto '" + message + "'");
        }
};
channel.basicConsume(QUEUE_NAME, true, consumer); // registra consumatore
```

Pattern Event-Driven Consumer

Il pattern Event-Driven Consumer risolve il problema delle applicazioni che necessitano di consumare messaggi non appena questi vengono consegnati.

Il **problema** è che se un consumatore controlla continuamente lo stato di un canale tramite un ciclo (polling), spreca cicli di processore o blocca il thread se non c'è nulla da leggere.

La **soluzione** è che anziché chiedere attivamente i messaggi, è il canale stesso a informare il consumatore quando un messaggio è disponibile. La consegna del messaggio funge da evento che avvia l'esecuzione.

Funzionamento

Il consumatore è un oggetto invocato dal sistema di messaggistica tramite una callback. Il suo ciclo di vita si divide in due fasi:

1. **Inizializzazione**: l'applicazione crea il consumatore, lo associa al canale e registra il metodo di callback tramite l'API. Il consumatore resta dormiente.
2. **Consumo**: quando il messaggio arriva, viene passato come parametro al metodo di callback del consumatore, il quale si "sveglia" e lo elabora.

Avvio

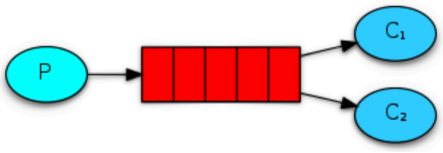
Per preparare l'ambiente ed eseguire il codice RabbitMQ (produttore e consumatore) in Java, è necessario seguire questi passi pratici:

1. **Librerie**: scaricare il broker (RabbitMQ server), le librerie client (amqp-client.jar) e le librerie per la gestione dei log (slf4j-api.jar).
2. **Compilazione**: compilare i file Java (Send.java, Recv.java) includendo il jar di RabbitMQ nel classpath usando il comando javac -cp amqp-client-4.0.4.jar Send.java Recv.java.

3. **Avvio server**: far partire il broker eseguendo sbin/rabbitmq-server.
4. **Esecuzione**: avviare i programmi compilati passando tutte le librerie necessarie al classpath tramite il comando java -cp .:amqp-client-4.0.2.jar:slf4j-api-1.7.21.jar:slf4j-simple-1.7.22.jar seguito dal nome della classe (Send o Recv).

Esempio 2

Il seguente esempio illustra un'architettura in cui un **singolo produttore (P)** invia messaggi a una determinata coda, alla quale sono collegati **due consumatori (C1 e C2)**. Il sistema fa in modo che i due consumatori vengano avvisati a turno della presenza di nuovi messaggi.



Con due consumatori

Collegando più consumatori alla stessa coda, l'esecuzione dei loro metodi di callback avviene in modalità round-robin man mano che i messaggi inviati dal produttore arrivano. A livello di codice, questo si ottiene istanziando due oggetti distinti (es. consumer1 e consumer2, estendendo DefaultConsumer), sovrascrivendo per ciascuno il metodo handleDelivery per definirne il comportamento, e infine registrandoli entrambi sulla stessa coda tramite due chiamate distinte a channel.basicConsume().

```
Consumer consumer1 = new DefaultConsumer(channel) {
        @Override
        public void handleDelivery(String consumerTag, Envelope envelope,
                        AMQP.BasicProperties properties, byte[] body) thro
ws IOException {
                String message = new String(body, "UTF-8");
                System.out.println("Ricevente uno '" + message + "'");
        }
};
channel.basicConsume(QUEUE_NAME, true, consumer1);

Consumer consumer2 = new DefaultConsumer(channel) {
        @Override
        public void handleDelivery(String consumerTag, Envelope envelope,
                        AMQP.BasicProperties properties, byte[] body) thro
ws IOException {
                String message = new String(body, "UTF-8");
                System.out.println("Ricevente due '" + message + "'");
        }
};
channel.basicConsume(QUEUE_NAME, true, consumer2);
```

Gestione Ack

La gestione manuale degli ack (ricevute) serve ad evitare il sovraccarico dei consumatori lenti. L'obiettivo e fare in modo che un consumatore che elabora i messaggi lentamente (e quindi manda gli ack in ritardo) riceva meno messaggi rispetto a un consumatore più veloce. Per implementare questo meccanismo:

1. si disabilita l'invio automatico dell'ack passando il valore false come secondo parametro nel metodo di registrazione:

   ```
   channel.basicConsume(QUEUE_NAME, false, consumer);
   ```

2. si imposta il canale affinché invii un solo messaggio alla volta (o un numero limitato) usando:

   ```
   channel.basicQos(1)
   ```

   In questo modo, RabbitMQ non manderà ulteriori messaggi al consumatore finché questo non ha confermato il precedente.

3. alla fine del lavoro di elaborazione (all'interno del metodo di callback), il consumatore invia esplicitamente l'ack richiamando:

   ```
   channel.basicAck(envelope.getDeliveryTag(), false);
   ```

Pattern Competing Consumers

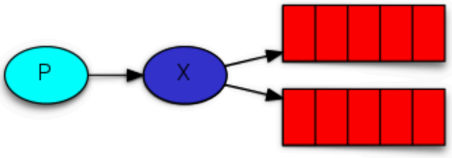
Questo pattern viene utilizzato quando un'applicazione non riesce a elaborare i messaggi tanto velocemente da stare al passo con la quantità in arrivo sul canale.

Il **problema** è che se il consumo è lento, i messaggi si accumulano (situazione che può derivare da tanti trasmettitori sullo stesso canale o da malfunzionamenti di rete) e canali equivalenti potrebbero ricevere dati in modo non uniforme.

La **soluzione** è creare dei consumatori competitivi su un singolo canale, i quali prendono ed elaborano i messaggi in modo concorrente (cioè in parallelo). Nella pratica, è il sistema di messaggistica stesso a determinare quale dei consumatori in ascolto riceverà il messaggio.

Esempio 3

Questo esempio architetturale illustra uno scenario in cui un produttore (P) manda messaggi a uno scambio X (exchange) anziché ad una singola coda. A questo scambio sono collegate due code distinte. Di conseguenza, ciascun consumatore collegato a tali code riceverà i messaggi.



Scambi

Negli scenari più avanzati, il produttore non invia i messaggi direttamente a una coda, ma li invia a uno "scambio" (exchange).

Funzionamento di uno scambio

Uno scambio è un intermediario che da un lato riceve i messaggi dai produttori e dall'altro lato li smista (manda) alle code.

Regole di smistamento

Lo scambio sa come gestire un messaggio ricevuto in base alle regole definite per il suo tipo di scambio. I tipi di scambio sono: direct, topic, headers e fanout. Fanout si limita a mandare i messaggi che riceve a tutte le code che conosce (come in un collegamento broadcast).

Nei modelli base, al contrario, si utilizza lo "scambio di deafault" identificato tramite una stringa vuota ("").

```
channel.basicPublish("", QUEUE_NAME, null, message.getBytes());
```

Creazione e uso di scambi

L'implementazione tecnica prevede una serie di passaggi e metodi specifici:

1. **Creazione**: si dichiara (crea) lo scambio specificandone il nome e il tipo (ad esempio FANOUT):

   ```
   String EXCHANGE_NAME = "logs";
   channel.exchangeDeclare(EXCHANGE_NAME, BuiltinExchangeType.FANOUT);
   ```

2. **Pubblicazione**: il produttore pubblica il messaggio indicando il nome dello scambio:

   ```
   String message = "Hello World!";
   channel.basicPublish(EXCHANGE_NAME, "", null, message.getBytes("UTF-
   8"));
   ```

3. **Gestione della coda**: il consumatore crea una coda vuota (che sarà non-durable, exclusive e auto-deleted). Il nome di questa coda è generato in modo random dal server, poiché non è necessario che il produttore lo conosca:

   ```
   String queueName = channel.queueDeclare().getQueue();
   ```

4. **Binding** (connessione): per far viaggiare i messaggi dallo scambio fino alla coda appena creata, è necessario stabilire un collegamento esplicito:

   ```
   channel.queueBind(queueName, EXCHANGE_NAME, “”);
   ```

5. **Ascolto**: si registra infine il consumatore sulla coda per permettergli di iniziare a ricevere i messaggi:

   ```
   channel.basicConsume(queueName, true, consumer);
   ```

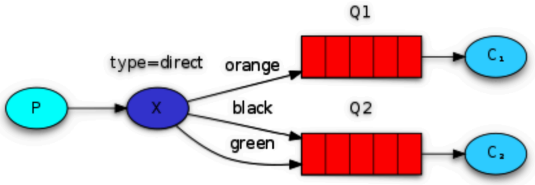
Design Pattern Publish-Subscribe

L'intento di questo pattern è usare i messaggi per annunciare eventi. Viene applicato per risolvere il problema di un produttore che deve inviare un evento a tutti i riceventi interessati, con la necessità che ogni subscriber venga notificato una sola volta per ciascun evento. L'evento è considerato

consumato, e può sparire dal canale, solo quando tutti i subscriber sono stati notificati. La soluzione consiste nell'inviare l'evento a un canale Publish-Subscribe, il quale provvede a mandare una copia di ciascun evento a ciascun ricevente. Architetturalmente, il canale in ingresso si divide in tanti canali di uscita quanti sono i subscriber. Questo pattern risulta ottimo per intercettare i messaggi, ad esempio per operazioni di debug, poiché permette di registrare i log senza dover cambiare il codice dell'applicazione.

Esempio 4

Questo esempio illustra un'architettura in cui un produttore manda vari tipi di messaggi. Il consumatore, collegandosi ad una specifica coda, è in grado di ricevere solo i messaggi a cui è interessato, poiché questi vengono filtrati in base al tipo.



Message Filter con RabbitMQ

Per implementare il filtraggio dei messaggi in RabbitMQ, è necessario seguire una specifica procedura sia lato produttore che lato consumatore:

- **Scambio**: durante la creazione dello scambio (exchange), bisogna indicare il tipo "direct" utilizzando il parametro BuiltinExchangeType.DIRECT:

  ```
  channel.exchangeDeclare(EXCHANGE_NAME, BuiltinExchangeType.DIRECT);
  ```

- **Produttore**: quando pubblica il messaggio tramite il metodo basicPublish(), il produttore deve indicare una routing key (una chiave di instradamento, ad esempio la stringa "cmd") come secondo parametro:

  ```
  channel.basicPublish(EXCHANGE_NAME, "cmd", null, msg.getBytes("UTF-
  8"));
  ```

- **Consumatore**: dopo aver creato una coda, il consumatore la connette allo scambio usando il metodo queueBind(), inserendo la stessa routing key come terzo parametro. Registrandosi poi con il metodo basicConsume(), il consumatore riceverà dalla coda esclusivamente i messaggi filtrati secondo la routing key che ha fornito:

  ```
  String queueName = channel.queueDeclare().getQueue();
  channel.queueBind(queueName, EXCHANGE_NAME, "cmd");

  channel.basicConsume(queueName, true, consumer);
  ```

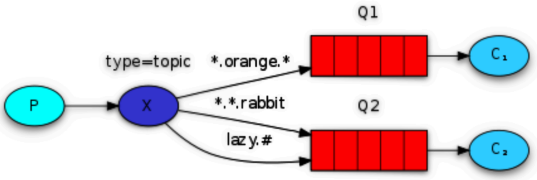
Design Pattern Message Filter

L'intento di questo pattern è far si che un componente riceva solo i messaggi a cui è interessato. Nasce per gestire il problema delle applicazioni in cui un produttore manda vari messaggi di notifica

per comunicazioni di diversa natura, ma alcuni consumatori necessitano solo di certi messaggi specifici. La soluzione consiste nell'utilizzare un filtro di messaggi che elimina i messaggi indesiderati da un canale basandosi su un determinato criterio (ad esempio, facendo passare solo messaggi di priorità alta).

Esempio 5

Il seguente esempio illustra lo scenario in cui un produttore manda messaggi caratterizzati da varie chiavi. I consumatori ricevono solo i messaggi a cui sono interessati: ad esempio, un consumatore C1 vuole ricevere solo i messaggi con la chiave "orange", mentre un consumatore C2 vuole ricevere esclusivamente quelli con le chiavi "rabbit" e "lazy".



Topic Exchange

Il topic exchange permette di realizzare scenari come quello dell'esempio sopra.

- **Creazione**: in fase di creazione dello scambio, bisogna specificare il tipo topic:

```
channel.exchangeDeclare(EXCHANGE_NAME, BuiltinExchangeType.TOPIC);
```

- **Invio dal produttore**: il produttore pubblica il messaggio indicando una routing key (usa stringa). Questa chiave ha solitamente un formato specifico composto da parole separate da punti (ad esempio, "velocità.colore.specie", come "quick.orange.rabbit").

```
channel.basicPublish(EXCHANGE_NAME, routingKey, null,

message.getBytes("UTF-8"));
```

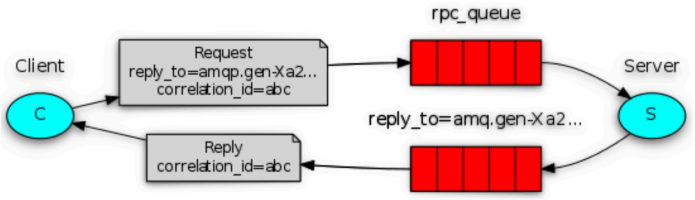
- **Ricezione dai consumatori**: i consumatori creano una coda e la collegano allo scambio fornendo una binding key (ad esempio, "**.orange.**").

```
String queueName = channel.queueDeclare().getQueue();
channel.queueBind(queueName, EXCHANGE_NAME, bindingKey);
```

- **Wildcard**: la potenza di questo scambio topic risiede nel fatto che la binding key può contenere dei caratteri speciali (le wildcard). Un asterisco (*) sostituisce esattamente una singola parola, mentre un cancelletto (#) sostituisce zero o più parole. Pertanto, C1 si connetterà usando la chiave **.orange.**, mentre C2 utilizzerà le chiavi **.*.rabbit* e lazy.#.

Esempio 6

Questo esempio illustra un'implementazione del modello RPC (Remote Procedure Call) su RabbitMQ. In questa architettura, un client effettua una richiesta specifica e si mette in attesa di ricevere una risposta dal server



Lato client: coda di callback

Per far sì che il modello RPC funzioni, il client che manda il messaggio deve poter ricevere la risposta.

→ **coda di risposta**: insieme alla richiesta, il client deve inviare al server l'indirizzo di una coda di callback (tramite la proprietà `replyTo`), indicando così dove il server dovrà recapitare il risultato.

→ **identificatore unico**: il client imposta anche un `correlationId`, ovvero una stringa contenente un identificatore univoco utile per ricollegare la futura risposta alla specifica richiesta inviata.

```
String replyQueueName = channel.queueDeclare().getQueue();
BasicProperties props = new BasicProperties
                                                        .Builder()
                                                        .correlationId(co
rrId)
                                                        .replyTo(replyQue
ueName)
                                                        .build();

channel.basicPublish("", reqQueueName, props, message.getBytes("UTF-8"));
```

→ **ascolto**: infine, il client registra un metodo di callback su questa coda di risposta (usando basicConsume()) per mettersi in attesa del risultato.

```
channel.basicConsume(replyQueueName, true, consumer);
```

Lato server

Il server ha il compito di leggere le richieste, elaborarle e restituire il risultato al mittente corretto.

→ quando elabora il messaggio, il server crea le proprietà della risposta inserendo lo stesso `correlationId` che ha ricevuto nella richiesta.

→ successivamente, pubblica la risposta inviandola alla coda del client (estraendo l'indirizzo dal campo `replyTo` fornito originariamente dal client).

→ come ultimo passo, il server invia un ack (ricevuta) per confermare che il messaggio di richiesta è stato gestito.

```
public void handleDelivery(String consumerTag, Envelope envelope,
                           BasicProperties properties, byte[] body) throw
s IOException {

        BasicProperties replyProps = new BasicProperties
                                        .Builder()
```

```
                                        .correlationId(properties.getCorre
lationId())
                                        .build();

        channel.basicPublish("", properties.getReplyTo(), replyProps,

response.getBytes("UTF-8"));

        channel.basicAck(envelope.getDeliveryTag(), false);
}
```

Operazioni sulla coda

Oltre alla semplice lettura e scrittura, RabbitMQ permette di effettuare alcune operazioni di gestione dirette sulle code:

- **Svuotamento**: è possibile eliminare tutto il contenuto attualmente presente in una coda (senza cancellare la coda stessa) invocando il metodo:

  ```
  channel.queuePurge("queue-name");
  ```

- **Cancellazione**: è possibile cancellare interamente una coda usando:

  ```
  channel.queueDelete("queue-name");
  ```

- **Cancellazione condizionale**: per evitare perdite di dati o interruzioni anomale, la cancellazione può essere vincolata. Si può decidere di eliminare la coda passando specifici parametri:
  - eliminare la coda solo se è vuota:

    ```
    channel.queueDelete("queue-name", false, true);
    ```

  - eliminare la coda solo se non è attualmente usata da nessun consumatore:

    ```
    channel.queueDelete("queue-name", true, false);
    ```

Messaggi che scadono

Il sistema consente di pubblicare un messaggio impostandolo affinché si autodistrugga dopo un determinato lasso di tempo (timeout). Questa funzionalità si attiva durante la pubblicazione tramite il metodo basicPublish, configurando la proprietà `expiration` (ad esempio passandogli il valore "60000" millisecondi) all'interno dell'oggetto BasicProperties associato al messaggio.

```
channel.basicPublish("exchangeName", "routingKey",
                      new BasicProperties.Builder()
                                                                .expiration
("60000")
```

```
                                                                    .build(),
                        "message".getBytes());
```

Pull di messaggi

Mentre il pattern standard prevede che i messaggi vengano “spinti” (push) verso il consumatore, esiste anche la possibilità per il consumatore di prelevare (pull) esplicitamente un singolo messaggio dalla coda a proprio piacimento.

→ questo si ottiene invocando il metodo basicGet(queueName, true) sul canale:

```
GetResponse response = channel.basicGet(queueName, true);
```

→ il metodo restituisce un’istanza di GetResponse. Se questa non è nulla, il consumatore può interrogarla per leggere l’header (le proprietà) e il body (il contenuto) del messaggio estratto.

```
if (null != response) {
        BasicProperties props = response.getProps();
        byte[] body = response.getBody();
        long deliveryTag = response.getEnvelope().getDeliveryTag();
        String message = new String(body, "UTF-8");
}
```

Recupero da fallimenti della rete

Poiché la rete può subire interruzioni, i client Java di RabbitMQ sono progettati per gestire le cadute di connessione in modo robusto.

- **Recupero automatico**: se la connessione fallisce, i client supportano il recupero automatico sia della connessione stessa, sia dell’intera topologia (ovvero ricreano automaticamente code, scambi, binding e consumatori).
- **Attivazione**: questo processo di recupero è abilitato di default (automaticamente).
- **Innesco**: il meccanismo scatta e si avvia non appena il sistema rileva un’eccezione di I/O, oppure quando un’operazione di lettura dalla socket incontra un timeout.